

Brainbase Model Product — Development Resource Index

Purpose

This document lists every site, official documentation page, repo, paper, and resource category that can be used to build or extend the Brainbase model product.

The product is externally exposed as first-party Brainbase models such as:

```
brainbase-chat  
brainbase-code  
brainbase-agent  
brainbase-fast  
brainbase-premium
```

Internally, it uses:

```
RouterPolicy  
RoutePlan  
Model Resolver  
Model Registry  
Policy Registry  
Portkey Executor  
OpenRouter model access  
Supabase traces/evals/artifacts
```

This document is not saying every resource is required on day one.

It separates resources into:

1. Required for MVP
2. Useful for implementation
3. Router/research resources
4. Benchmarking resources
5. Optional future infrastructure
6. Optional future training/fine-tuning resources

1. Required MVP Resources

These are the only external platforms needed for the first shippable version.

1.1 Portkey

Purpose

Portkey is the execution gateway. Brainbase decides the route internally, then Portkey executes the selected model.

Used for

```
model execution
OpenRouter integration
gateway-level observability
provider key management
optional fallbacks/retries/rate limits later
```

Resource

```
https://portkey.ai/docs/product/ai-gateway
https://portkey.ai/docs/integrations/llms/openrouter
https://portkey.ai/docs/product/observability
```

Required key

```
PORTKEY_API_KEY=
```

1.2 OpenRouter

Purpose

OpenRouter is the broad model marketplace/source. It lets Brainbase access many open-source, open-weight, and closed-source models through model strings.

Used for

```
model string passthrough
accessing many providers/models quickly
testing new models
listing available model slugs
initial model pool
```

Resource

```
https://openrouter.ai/docs/quickstart
https://openrouter.ai/models
https://openrouter.ai/docs
```

Required key

```
OPENROUTER_API_KEY=
```

1.3 Supabase

Purpose

Supabase is the database and lightweight storage layer for the MVP.

Used for

```
model registry
policy registry
public model aliases
request traces
benchmark results
router artifacts
eval reports
training dataset exports
```

Resources

```
https://supabase.com/docs/guides/getting-started/api-keys
https://supabase.com/docs/guides/storage
https://supabase.com/docs
```

Required values

```
SUPABASE_URL=  
SUPABASE_SECRET_KEY=  
SUPABASE_PUBLISHABLE_KEY=
```

Legacy equivalent names:

```
SUPABASE_URL=  
SUPABASE_SERVICE_ROLE_KEY=  
SUPABASE_ANON_KEY=
```

2. Required Development Framework Resources

These are implementation resources for the backend.

2.1 FastAPI

Purpose

FastAPI should be used to build the OpenAI-compatible API layer.

Used for

```
/v1/chat/completions  
/v1/models  
health checks  
internal debug endpoints  
benchmark endpoints  
policy/model registry endpoints
```

Resource

```
https://fastapi.tiangolo.com/
```

2.2 Pydantic

Purpose

Pydantic should define and validate the internal protocol objects.

Used for

```
RouterRequest  
ModelProfile  
RoutingContext  
RoutingBudget  
RoutePlan  
RouteStep  
FeedbackEvent  
request validation  
response validation
```

Resource

```
https://docs.pydantic.dev/
```

2.3 Uvicorn

Purpose

Uvicorn runs the FastAPI app locally and in production.

Used for

```
local development server  
production ASGI server  
Render deployment command
```

Resource

```
https://www.uvicorn.org/
```

2.4 OpenAI Python SDK

Purpose

The product should be compatible with OpenAI-style SDK usage.

Customers should be able to do:

```
from openai import OpenAI

client = OpenAI(
    api_key="...",
    base_url="https://api.brainbase.ai/v1"
)

client.chat.completions.create(
    model="brainbase-chat",
    messages=[...]
```

Resources

```
https://github.com/openai/openai-python
https://platform.openai.com/docs/api-reference/chat
```

3. Deployment and Developer Workflow Resources

3.1 Render

Purpose

Render is the recommended MVP deployment target.

Used for

```
hosting FastAPI backend
setting environment variables
quick deployment from GitHub
Docker-based deployment
```

Resources

```
https://render.com/docs/configure-environment-variables  
https://render.com/docs/blueprint-spec  
https://render.com/docs/docker
```

Optional key if coding agent deploys automatically

```
RENDER_API_KEY=
```

If deployment is manual through the Render UI, the agent does not need this key.

3.2 GitHub

Purpose

GitHub is used for source control and optionally CI/CD.

Used for

```
repository hosting  
pull requests  
version control  
GitHub Actions later  
fine-grained PATs only if coding agent needs repo access
```

Resources

```
https://docs.github.com/en/actions  
https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/  
managing-your-personal-access-tokens
```

Optional key

```
GITHUB_TOKEN=
```

Only needed if the coding agent must push code, create PRs, or configure GitHub programmatically.

4. Core Product Concepts to Document Internally

These are not third-party sites, but they should exist as internal Brainbase docs.

4.1 Brainbase Router Protocol

Purpose

Defines the internal contract every router must implement.

Core interface

```
RouterPolicy.plan(...) -> RoutePlan
```

Internal docs to create

```
docs/protocol/router-policy.md  
docs/protocol/route-plan.md  
docs/protocol/model-profile.md  
docs/protocol/feedback-event.md
```

4.2 Model Resolver

Purpose

Allows routers to output model strings like:

```
openrouter:anthropic/claude-sonnet-4.6  
openrouter:qwen/qwen-coder  
anthropic:claude-opus-latest  
openai:gpt-latest  
selfhosted:brainbase-qwen-agent-v1
```

Then the resolver converts them into executor-specific format.

Internal docs to create


```
docs/model-resolver/model-id-format.md
docs/model-resolver/openrouter-passthrough.md
docs/model-resolver/dynamic-model-testing.md
```

4.3 Policy Registry

Purpose

Tracks available router policies.

Examples:

```
always-strongest:v0
always-cheapest:v0
hydra:v1
graphrouter:v1
brainbase-trained:v3
```

Internal docs to create

```
docs/registries/policy-registry.md
docs/registries/policy-lifecycle.md
```

4.4 Model Registry

Purpose

Tracks all models available to the runtime.

Internal docs to create

```
docs/registries/model-registry.md
docs/registries/model-lifecycle.md
docs/registries/model-calibration.md
```

5. Router / Algorithm Research Resources

These are papers, repos, and frameworks that can be implemented later as RouterPolicy plug-ins.

The stable product shell should not depend directly on any of these. Each one should be wrapped behind:

```
RouterPolicy.plan(...) -> RoutePlan
```

5.1 HyDRA

Full name

HyDRA: Hybrid Dynamic Routing Architecture for Heterogeneous LLM Pools

Why it matters

HyDRA is the most directly aligned with the Brainbase product design because it predicts required capabilities and matches them against config-defined model profiles.

How it fits

```
RouterPolicy plug-in  
mode: single  
requires: model capability profiles  
good for: configurable model catalog, new model additions
```

Resource

```
https://arxiv.org/abs/2605.17106
```

5.2 GraphRouter

Full name

GraphRouter: A Graph-based Router for LLM Selections

Why it matters

GraphRouter models interactions between tasks, queries, and LLMs as a graph. It is relevant as a stronger routing baseline and possible future policy.

How it fits

```
RouterPolicy plug-in  
mode: single  
requires: graph/index artifacts  
good for: effect-cost modeling, model selection
```

Resources

```
https://arxiv.org/abs/2410.03834  
https://github.com/ulab-uiuc/GraphRouter
```

5.3 Avengers-Pro

Full name

Beyond GPT-5: Making LLMs Cheaper and Better via Performance-Efficiency Optimized Routing

Why it matters

Avengers-Pro is one of the strongest examples of a router being treated like a model-level system and compared to frontier models.

How it fits

```
RouterPolicy plug-in  
mode: single  
requires: embedding/clustering artifacts  
good for: performance-efficiency tradeoffs
```

Resources

```
https://arxiv.org/abs/2508.12631  
https://github.com/ZhangYiqun018/AvengersPro
```

5.4 LLMRouterBench

Why it matters

LLMRouterBench is a benchmark and unified framework for LLM routing. It should influence Brainbase's benchmark runner.

How it fits

eval framework inspiration
benchmark adapter
not necessarily production policy

Resources

<https://arxiv.org/abs/2601.07206>
<https://github.com/ynulihao/LLMRouterBench>

5.5 LLMRouter Library

Why it matters

Earlier in the discussion, this was considered as a routing library/baseline source. It is useful as a reference for router implementations and possible baselines.

How it fits

baseline source
research framework
possible policy adapter

Resource

<https://github.com/ulab-uiuc/LLMRouter>

5.6 RouteLLM

Why it matters

RouteLLM is a practical serving/evaluation framework for strong-vs-weak routing. It is a useful baseline but should not define the whole product architecture.

How it fits

```
baseline policy  
evaluation reference  
possible MFRouter-style adapter
```

Resource

```
https://github.com/lm-sys/RouteLLM
```

5.7 RouteNLP

Full name

RouteNLP: Closed-Loop LLM Routing with Conformal Cascading and Distillation Co-Optimization

Why it matters

RouteNLP is relevant for cost reduction, cascade routing, confidence thresholds, and future distillation loops.

How it fits

```
RouterPolicy plug-in  
mode: cascade  
requires: confidence/calibration  
good for: enterprise workflows, cost optimization, distillation loop
```

Resource

```
https://arxiv.org/abs/2604.23577
```

5.8 TwinRouterBench

Why it matters

TwinRouterBench is important because Brainbase's Kafka/agent workflows are multi-step. It benchmarks routing at intermediate agent steps.

How it fits

```
benchmark inspiration  
agent-step routing evaluation  
workflow-level scoring
```

Resources

```
https://arxiv.org/abs/2605.18859  
https://github.com/CommonstackAI/TwinRouterBench
```

5.9 MTRouter

Full name

MTRouter: Cost-Aware Multi-Turn LLM Routing with History-Model Joint Embeddings

Why it matters

MTRouter is relevant for multi-turn and workflow-history-aware routing.

How it fits

```
RouterPolicy plug-in  
mode: agent_step  
requires: conversation/workflow history  
good for: long-horizon agents
```

Resources

```
https://arxiv.org/abs/2604.23530  
https://github.com/ZhangYiqun018/MTRouter
```

5.10 R2-Router

Why it matters

R2-Router routes not only the model but also the output-length budget.

How it fits

```
RouterPolicy plug-in  
mode: budgeted_single  
requires: generation budget support  
good for: cost-sensitive outputs
```

Resource

```
https://arxiv.org/abs/2602.02823
```

5.11 OrcaRouter

Why it matters

OrcaRouter is relevant for hybrid offline-online learning and contextual bandit routing.

How it fits

```
RouterPolicy plug-in  
optional interface: OnlineLearningRouterPolicy.update()  
requires: feedback events  
good for: adaptive production routing
```

Resource

```
https://arxiv.org/abs/2605.30736
```

5.12 BaRP

Full name

Learning to Route LLMs from Bandit Feedback: One Policy, Many Trade-offs

Why it matters

BaRP is relevant for bandit-feedback routing and preference-tunable cost/quality tradeoffs.

How it fits

```
RouterPolicy plug-in  
optional online-learning interface  
requires: partial-feedback handling  
good for: adaptive tenant-level tradeoffs
```

Resource

```
https://arxiv.org/abs/2510.07429
```

5.13 Router-R1

Why it matters

Router-R1 is relevant for multi-round routing and aggregation, where a router can call multiple models and combine outputs.

How it fits

```
RouterPolicy plug-in  
mode: multi_call  
requires: workflow executor  
good for: best-quality mode, hard reasoning, model aggregation
```

Resources

```
https://arxiv.org/abs/2506.09033  
https://github.com/ulab-uiuc/Router-R1
```

5.14 RouteJudge / ORBIT

Why it matters

RouteJudge and ORBIT are relevant for preference-aware evaluation and standardized router benchmarking.

How it fits

```
benchmark inspiration  
evaluation adapter  
preference-based router scoring
```

Resources

```
https://arxiv.org/abs/2606.18774  
https://routejudge.cn  
https://github.com/AIGNLAI/LAMDA-ORBIT
```

5.15 DecoR / CodaSet

Full name

Beyond Query Memorization: Large Language Model Routing with Query Decomposition and Historical Matching

Why it matters

DecoR is relevant for OOD/generalization and avoiding query memorization.

How it fits

```
RouterPolicy plug-in  
mode: single  
requires: historical matching  
good for: customer/domain-specific logs
```

Resources

```
https://arxiv.org/abs/2605.25558  
https://github.com/lvbotenbest/DecoR
```

5.16 Policy-Guided Stepwise Model Routing

Why it matters

This is relevant for chain-of-thought/intermediate-step routing during reasoning tasks.

How it fits

```
RouterPolicy plug-in  
mode: agent_step or stepwise_reasoning  
requires: intermediate reasoning-state interface  
good for: math/reasoning workflows
```

Resource

```
https://arxiv.org/abs/2605.06116
```

5.17 Lookahead Routing

Why it matters

Lookahead Routing predicts representations of potential outputs before full inference and uses them for routing.

How it fits

```
RouterPolicy plug-in  
mode: single  
requires: lookahead feature model  
good for: complex/ambiguous queries
```

Resources

<https://arxiv.org/abs/2510.19506>
<https://github.com/huangcb01/lookahead-routing>

5.18 TRouter

Full name

Task-Aware LLM Routing with Multi-Level Task-Profile-Guided Data Synthesis for Cold-Start Scenarios

Why it matters

TRouter is relevant when Brainbase has no in-domain data for a new customer or task.

How it fits

RouterPolicy plug-in
mode: single
requires: task taxonomy / synthetic data
good for: cold-start customer routing

Resource

<https://arxiv.org/abs/2604.09377>

5.19 GMTRouter

Why it matters

GMTRouter is relevant for personalized routing over multi-turn user interactions.

How it fits

RouterPolicy plug-in
mode: agent_step or personalized_single
requires: tenant/user preference context
good for: customer-specific personalization

Resources

```
https://arxiv.org/abs/2511.08590
https://github.com/ulab-uiuc/GMTRouter
```

5.20 RCR-Router

Why it matters

RCR-Router is relevant for context/memory routing in multi-agent systems.

How it fits

```
RouterPolicy plug-in
mode: context_routing
requires: structured memory store
good for: agent workflows and context reduction
```

Resource

```
https://arxiv.org/abs/2508.04903
```

5.21 BoundaryRouter / RouteBench

Why it matters

BoundaryRouter is relevant to deciding whether a query should be handled by direct LLM inference or escalated to a full agent.

How it fits

```
RouterPolicy plug-in
mode: agent_or_direct
requires: agent execution metadata
good for: Brainbase agent orchestration
```

Resource

<https://arxiv.org/abs/2605.07180>

6. Optional Provider/Gateway Resources

6.1 LiteLLM

Purpose

LiteLLM was discussed as an alternative execution gateway to Portkey.

Current decision

Not required for MVP because Brainbase already uses Portkey and Portkey can integrate with OpenRouter.

When to revisit

if Portkey blocks model-string passthrough
if Portkey makes router benchmarking harder
if direct provider support becomes easier through LiteLLM
if we want a thinner open-source gateway

Resources

https://docs.litellm.ai/docs/proxy/quick_start
<https://docs.litellm.ai/docs/providers/openrouter>

7. Optional Self-Hosted Model Resources

These are not required for MVP, but are important for the long-term goal of supporting every possible model.

7.1 vLLM

Purpose

High-throughput serving for open-source/self-hosted models.

Used for later

```
self-hosted Brainbase fine-tuned models  
OpenAI-compatible local endpoints  
custom model serving
```

Resource

```
https://docs.vllm.ai/
```

7.2 SGLang

Purpose

Serving/runtime framework for efficient LLM programs and models.

Used for later

```
self-hosted models  
structured generation  
advanced model serving
```

Resource

```
https://docs.sglang.ai/
```

7.3 Ollama

Purpose

Local model runtime for quick experimentation with open models.

Used for later

```
local testing
developer experimentation
possibly local model lane
```

Resource

```
https://github.com/ollama/ollama
```

7.4 Hugging Face Hub

Purpose

Model/artifact source for open models and fine-tuned model artifacts.

Used for later

```
model downloads
fine-tuned model storage
open-source model discovery
tokenizer/model artifacts
```

Resource

```
https://huggingface.co/docs/huggingface\_hub/index
```

8. Optional Direct Provider Resources

These are not required for the MVP if all execution goes through Portkey + OpenRouter.

Add them later if Brainbase needs direct pricing, provider-specific features, lower latency, or fallback outside OpenRouter.

8.1 OpenAI Direct

```
https://platform.openai.com/docs  
https://platform.openai.com/docs/api-reference  
https://github.com/openai/openai-python
```

Possible key:

```
OPENAI_API_KEY=
```

8.2 Anthropic Direct

```
https://docs.anthropic.com/
```

Possible key:

```
ANTHROPIC_API_KEY=
```

8.3 Google Gemini Direct

```
https://ai.google.dev/gemini-api/docs
```

Possible key:

```
GEMINI_API_KEY=  
GOOGLE_API_KEY=
```

8.4 Google Vertex AI

```
https://cloud.google.com/vertex-ai/generative-ai/docs
```

Possible values:


```
GOOGLE_APPLICATION_CREDENTIALS=  
GOOGLE_CLOUD_PROJECT=  
GOOGLE_CLOUD_LOCATION=
```

8.5 Azure OpenAI

```
https://learn.microsoft.com/azure/ai-services/openai/
```

Possible values:

```
AZURE_OPENAI_API_KEY=  
AZURE_OPENAI_ENDPOINT=  
AZURE_OPENAI_API_VERSION=
```

8.6 AWS Bedrock

```
https://docs.aws.amazon.com/bedrock/
```

Possible values:

```
AWS_ACCESS_KEY_ID=  
AWS_SECRET_ACCESS_KEY=  
AWS_REGION=
```

9. Optional Infrastructure Resources

9.1 AWS S3

Purpose

Only needed later if Supabase Storage is not enough.

Used for

```
router artifacts
training datasets
large trace exports
benchmark files
model outputs
```

Resource

```
https://docs.aws.amazon.com/s3/
```

Possible values:

```
S3_BUCKET=
S3_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
```

9.2 Cloudflare R2

Purpose

Alternative to S3.

Resource

```
https://developers.cloudflare.com/r2/
```

Possible values:

```
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=
R2_ENDPOINT_URL=
```

9.3 Redis / Upstash

Purpose

Not needed for MVP. Useful later for rate limits, locks, caching, and coordination.

Resources

```
https://redis.io/docs/latest/  
https://upstash.com/docs/redis
```

Possible values:

```
REDIS_URL=  
UPSTASH_REDIS_REST_URL=  
UPSTASH_REDIS_REST_TOKEN=
```

9.4 Kafka

Purpose

Not needed for standalone MVP. Relevant when integrating with Brainbase agent orchestration.

Resource

```
https://kafka.apache.org/documentation/  
https://docs.confluent.io/
```

Possible values:

```
KAFKA_BOOTSTRAP_SERVERS=  
KAFKA_API_KEY=  
KAFKA_API_SECRET=  
KAFKA_SCHEMA_REGISTRY_URL=  
KAFKA_SCHEMA_REGISTRY_API_KEY=  
KAFKA_SCHEMA_REGISTRY_API_SECRET=
```

10. Optional Observability Resources

10.1 Portkey Observability

Current recommendation

Use Portkey's built-in observability first. Do not add LangFuse/Sentry unless needed.

Resource

```
https://portkey.ai/docs/product/observability
```

10.2 Sentry

Purpose

Optional later for app-level error tracking.

```
https://docs.sentry.io/
```

Possible value:

```
SENTRY_DSN=
```

10.3 LangFuse

Purpose

Optional. Not needed for lean MVP because Portkey + Supabase traces should cover the core need.

```
https://langfuse.com/docs
```

Possible values:

```
LANGFUSE_PUBLIC_KEY=  
LANGFUSE_SECRET_KEY=  
LANGFUSE_HOST=
```

11. Optional Training / Fine-Tuning Resources

These are not needed for MVP, but are relevant when Brainbase starts training routers or fine-tuning models.

11.1 Hugging Face

```
https://huggingface.co/docs  
https://huggingface.co/docs/huggingface_hub/index
```

Possible value:

```
HUGGINGFACE_TOKEN=
```

11.2 Weights & Biases

```
https://docs.wandb.ai/
```

Possible value:

```
WANDB_API_KEY=
```

11.3 Modal

```
https://modal.com/docs
```

Possible values:

```
MODAL_TOKEN_ID=  
MODAL_TOKEN_SECRET=
```

11.4 RunPod

```
https://docs.runpod.io/
```

Possible value:

```
RUNPOD_API_KEY=
```

11.5 CoreWeave

```
https://docs.coreweave.com/
```

Possible value:

```
COREWEAVE_API_TOKEN=
```

12. Required MVP Environment Variables

For the lean product shell:

```
PORTKEY_API_KEY=  
  
OPENROUTER_API_KEY=  
  
SUPABASE_URL=  
SUPABASE_SECRET_KEY=  
SUPABASE_PUBLISHABLE_KEY=
```

If using legacy Supabase names:

```
SUPABASE_URL=  
SUPABASE_SERVICE_ROLE_KEY=  
SUPABASE_ANON_KEY=
```

If the coding agent must deploy to Render:

```
RENDER_API_KEY=
```

If the coding agent must push to GitHub:

```
GITHUB_TOKEN=
```

13. Recommended MVP Stack

```
Backend:  
  Python  
  FastAPI  
  Pydantic  
  Uvicorn  
  
Execution:  
  Portkey  
  
Model access:  
  OpenRouter through Portkey  
  
Database:  
  Supabase Postgres  
  
Artifact/report storage:  
  Supabase Storage  
  
Deployment:  
  Render  
  
Version control:  
  GitHub  
  
Public API style:  
  OpenAI-compatible chat completions
```

```
Internal protocol:  
RouterPolicy.plan(...) -> RoutePlan
```

14. What to Avoid in MVP

Do not add these unless there is a clear immediate need:

```
LangFuse  
Sentry  
Redis  
Kafka  
Stripe  
Clerk  
AWS S3  
direct OpenAI API key  
direct Anthropic API key  
direct Gemini API key  
vLLM  
SGLang  
Ollama  
W&B  
RunPod  
Modal
```

These are useful later, but they are not required to prove the product shell.

15. Final Implementation Resource Priority

Must-read first

```
Portkey OpenRouter integration  
OpenRouter quickstart  
Supabase API keys  
Supabase Storage  
FastAPI docs  
Pydantic docs  
Render environment variables  
OpenAI Python SDK
```


Must-read for architecture

HyDRA
LLMRouterBench
GraphRouter
Avengers-Pro
RouteNLP
TwinRouterBench
MTRouter
Router-R1
R2-Router
OrcaRouter
RouteJudge / ORBIT

Must-read later

vLLM
SGLang
Ollama
Hugging Face Hub
LiteLLM
AWS S3 or Cloudflare R2
Kafka
Redis/Upstash
training/fine-tuning infra

16. Final Resource Summary

The platform should be built around a small core:

Portkey
OpenRouter
Supabase
FastAPI
Pydantic
Render
GitHub

The platform should be extensible toward:

```
HyDRA  
GraphRouter  
Avengers-Pro  
LLMRouterBench  
RouteNLP  
TwinRouterBench  
MTRouter  
OrcaRouter  
Router-R1  
R2-Router  
RouteJudge/ORBIT  
future router papers
```

The long-term model access layer can expand toward:

```
OpenRouter  
direct OpenAI  
direct Anthropic  
direct Gemini  
direct Bedrock  
direct Azure OpenAI  
vLLM  
SGLang  
Ollama  
custom Brainbase fine-tuned models
```

The product should stay stable while all of these plug in through:

```
RouterPolicy.plan(...) -> RoutePlan  
ModelRegistry  
ModelResolver  
PortkeyExecutor  
BenchmarkRunner  
ShadowTesting
```